

Problem Set 1 - Suggested Solutions

1. Sum Stats and Institutional Details

(a) Do some brief diligence on the products and industry. What do you anticipate may be some important determinants of demand, substitution, and pricing?

Answers may vary. Students may include information on conditions treated by the medications, the prevalence of these conditions, or relevant (perceived or otherwise) differences between the products. Responses may also include general information on popularity or use trends for the products.

(b) Complete the table above by adding columns for the mean: market share, unit price, price/100 tablets, and unit wholesale price. Interpret any notable patterns you see in the summary statistics.

```
In [1]: import numpy as np
import pandas as pd
from linearmodels.iv import IV2SLS
import statsmodels.api as sm
import statsmodels.formula.api as smf
otc = pd.read_csv('OTC_Sales.csv')
```

```
In [2]: # calculate total sales for each store-week pair
product_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
otc['total_sales'] = otc[[f'sales_{number}' for number in product_numbers]].
```

```
In [3]: # reshape to make it easier to work with
otc = pd.wide_to_long(otc, ['sales', 'price', 'cost', 'prom'], i = ['store',
summary = otc.groupby('product').agg(
    price = ('price', 'mean'),
    total_sales = ('sales', 'sum'),
    wholesale_price = ('cost', 'mean')
).reset_index()

# add given information so that table is easy to read
brand_names = {
    1: 'Tylenol',
    2: 'Tylenol',
    3: 'Tylenol',
    4: 'Advil',
    5: 'Advil',
    6: 'Advil',
    7: 'Bayer',
```

```

    8: 'Bayer',
    9: 'Bayer',
   10: 'Store Brand',
   11: 'Store Brand'}
size = {
    1: 25,
    2: 50,
    3: 100,
    4: 25,
    5: 50,
    6: 100,
    7: 25,
    8: 50,
    9: 100,
   10: 50,
   11: 100
}
summary['brand'] = summary['product'].map(brand_names)
summary['size'] = summary['product'].map(size)

# calculate new variables
summary['market_share'] = summary['total_sales'] / (summary['total_sales'].sum())
summary['price_per_100'] = (summary['price'] / summary['size']) * 100

# reorder columns and round for clarity
table_1 = summary[['product', 'brand', 'size', 'market_share', 'price', 'price_per_100', 'wholesale_price']]
table_1

```

Out[3]:

	product	brand	size	market_share	price	price_per_100	wholesale_price
0	1	Tylenol	25	0.14	3.43	13.71	2.19
1	2	Tylenol	50	0.18	4.95	9.89	3.68
2	3	Tylenol	100	0.12	7.03	7.03	5.77
3	4	Advil	25	0.12	2.97	11.88	2.03
4	5	Advil	50	0.08	5.15	10.29	3.63
5	6	Advil	100	0.04	8.16	8.16	6.10
6	7	Bayer	25	0.04	2.67	10.70	1.85
7	8	Bayer	50	0.03	3.62	7.24	2.44
8	9	Bayer	100	0.08	3.97	3.97	3.71
9	10	Store Brand	50	0.09	1.94	3.87	0.91
10	11	Store Brand	100	0.08	4.43	4.43	1.91

2. Logit Demand Estimation

Consider the utility function for product j in store-week t for consumer i :

$$u_{ijt} = -\alpha p_{jt} + X_{jt}\beta + \xi_{jt} + \epsilon_{ijt} \quad (1)$$

where p_{jt} is price, X_{jt} are other observed product characteristics, ξ_{jt} are unobserved product characteristics, and ϵ_{ijt} is an i.i.d. EV1 logit consumer-product-market unobservable.

Estimate this model:

(a) Using OLS with price, promotion, and an indicator for whether the product is a "store brand" as product characteristics.

```
In [4]: # need shares for each observation
otc['share'] = otc['sales'] / otc['count']
otc['outside_share'] = 1 - (otc['total_sales'] / otc['count'])

# indicator for store brand
otc['store_brand'] = np.where(otc['product'] > 9, 1, 0)
otc['size'] = otc['product'].map(size)

In [5]: # create dependent variable: ln(share - outside_share)
otc['ln_share'] = np.log(otc['share']) - np.log(otc['outside_share'])

In [6]: model = smf.ols('ln_share ~ price + prom + store_brand', data = otc)
logit_modela = model.fit()
logit_modela.summary()
```

Out [6]:

OLS Regression Results

Dep. Variable:	ln_share	R-squared:	0.026
Model:	OLS	Adj. R-squared:	0.026
Method:	Least Squares	F-statistic:	335.2
Date:	Mon, 17 Feb 2025	Prob (F-statistic):	8.60e-215
Time:	10:15:25	Log-Likelihood:	-48633.
No. Observations:	37290	AIC:	9.727e+04
Df Residuals:	37286	BIC:	9.731e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-7.6453	0.014	-552.117	0.000	-7.672	-7.618
price	-0.0662	0.003	-24.543	0.000	-0.071	-0.061
prom	0.1990	0.016	12.316	0.000	0.167	0.231
store_brand	-0.2672	0.013	-21.180	0.000	-0.292	-0.242

Omnibus:	2157.310	Durbin-Watson:	1.362
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2189.616
Skew:	-0.552	Prob(JB):	0.00
Kurtosis:	2.563	Cond. No.	18.2

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

(b) Using OLS with price and promotion as product characteristics and product fixed effects (where a "product" is a brand-size combination).

```
In [7]: model = smf.ols('ln_share ~ price + prom + C(product)', data = otc)
logit_modelb = model.fit()
logit_modelb.summary()
```

Out [7]:

OLS Regression Results

Dep. Variable:	ln_share	R-squared:	0.457
Model:	OLS	Adj. R-squared:	0.457
Method:	Least Squares	F-statistic:	2614.
Date:	Mon, 17 Feb 2025	Prob (F-statistic):	0.00
Time:	10:15:25	Log-Likelihood:	-37744.
No. Observations:	37290	AIC:	7.551e+04
Df Residuals:	37277	BIC:	7.562e+04
Df Model:	12		
Covariance Type:	nonrobust		
	coef	std err	t P> t [0.025 0.975]
Intercept	-6.0693	0.037	-164.587 0.000 -6.142 -5.997
C(product)[T.2]	0.6879	0.023	30.566 0.000 0.644 0.732
C(product)[T.3]	0.9188	0.040	22.778 0.000 0.840 0.998
C(product)[T.4]	-0.4312	0.017	-25.661 0.000 -0.464 -0.398
C(product)[T.5]	-0.1960	0.024	-8.178 0.000 -0.243 -0.149
C(product)[T.6]	0.0045	0.051	0.088 0.930 -0.096 0.105
C(product)[T.7]	-1.6597	0.018	-93.199 0.000 -1.695 -1.625
C(product)[T.8]	-1.5844	0.016	-96.398 0.000 -1.617 -1.552
C(product)[T.9]	-0.5523	0.018	-31.542 0.000 -0.587 -0.518
C(product)[T.10]	-1.2360	0.022	-56.168 0.000 -1.279 -1.193
C(product)[T.11]	-0.7387	0.019	-38.296 0.000 -0.777 -0.701
price	-0.3404	0.010	-33.375 0.000 -0.360 -0.320
prom	0.3220	0.013	25.508 0.000 0.297 0.347
Omnibus:	2680.344	Durbin-Watson:	1.901
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3703.376
Skew:	-0.621	Prob(JB):	0.00
Kurtosis:	3.916	Cond. No.	112.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

(c) Estimate the models of (a) and (b) using the Hausman instrument (average price in other markets).

```
In [8]: def get_instruments(df):
# compute average prices per store, week, and product
avg_prices = df.groupby(['store', 'week', 'product'])['price'].mean().re

# merge to bring average prices into the original dataframe
df = df.merge(avg_prices, on=['store', 'week', 'product'], suffixes=('',

# compute instrument: average price across stores excluding the current
df['instrument'] = df.groupby(['week', 'product'])['price_avg'].transform

return df.drop(columns=['price_avg'])

otc = get_instruments(otc)
```

```
In [9]: # model a using IV
formula = ('ln_share ~ 1 + prom + store_brand + [price ~ instrument]')
logit_modela_iv = IV2SLS.from_formula(formula, otc).fit()
logit_modela_iv.summary
```

Out [9]:

IV-2SLS Estimation Summary

Dep. Variable:	ln_share	R-squared:	0.0263
Estimator:	IV-2SLS	Adj. R-squared:	0.0262
No. Observations:	37290	F-statistic:	874.64
Date:	Mon, Feb 17 2025	P-value (F-stat)	0.0000
Time:	10:15:25	Distribution:	chi2(3)
Cov. Estimator:	robust		

Parameter Estimates

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	-7.6453	0.0143	-533.36	0.0000	-7.6734	-7.6172
prom	0.1990	0.0158	12.585	0.0000	0.1680	0.2300
store_brand	-0.2672	0.0136	-19.635	0.0000	-0.2938	-0.2405
price	-0.0662	0.0028	-23.364	0.0000	-0.0718	-0.0607

Endogenous: price

Instruments: instrument

Robust Covariance (Heteroskedastic)

Debiased: False

```
In [10]: # model b using IV
otc_p = pd.get_dummies(otc, columns = ['product'], drop_first = True)
fixed_effects = " + ".join([col for col in otc_p.columns if col.startswith("

formula = f'ln_share ~ 1 + prom + {fixed_effects} + [price ~ instrument]'
logit_modelb_iv = IV2SLS.from_formula(formula, otc_p).fit()
logit_modelb_iv.summary
```

Out[10]:

IV-2SLS Estimation Summary			
Dep. Variable:	ln_share	R-squared:	0.4520
Estimator:	IV-2SLS	Adj. R-squared:	0.4519
No. Observations:	37290	F-statistic:	4.151e+04
Date:	Mon, Feb 17 2025	P-value (F-stat)	0.0000
Time:	10:15:25	Distribution:	chi2(12)
Cov. Estimator:	robust		

Parameter Estimates						
	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	-5.4225	0.0531	-102.16	0.0000	-5.5265	-5.3184
prom	0.2648	0.0142	18.676	0.0000	0.2370	0.2926
product_2	0.9765	0.0253	38.530	0.0000	0.9269	1.0262
product_3	1.6005	0.0563	28.417	0.0000	1.4901	1.7109
product_4	-0.5144	0.0140	-36.804	0.0000	-0.5418	-0.4870
product_5	0.1302	0.0293	4.4397	0.0000	0.0727	0.1876
product_6	0.8989	0.0738	12.187	0.0000	0.7544	1.0435
product_7	-1.7945	0.0169	-105.96	0.0000	-1.8277	-1.7613
product_8	-1.5404	0.0145	-105.93	0.0000	-1.5689	-1.5119
product_9	-0.4379	0.0155	-28.302	0.0000	-0.4683	-0.4076
product_10	-1.5100	0.0278	-54.341	0.0000	-1.5644	-1.4555
product_11	-0.5457	0.0227	-24.046	0.0000	-0.5902	-0.5012
price	-0.5286	0.0153	-34.607	0.0000	-0.5586	-0.4987

Endogenous: price
Instruments: instrument
Robust Covariance (Heteroskedastic)
Debiased: False

(d) Compute the mean own-price elasticities for all products

Recall that $\eta_j = \frac{\partial q_j}{\partial p_j} \frac{p_j}{q_j} = \alpha p_j (1 - s_j)$

```
In [11]: alpha = logit_modelb_iv.params.price
otc['elasticity'] = alpha * (otc['price']) * (1 - otc['share'])

# add this to our summary table so that we can easily compare
sum_elast = otc.groupby('product').agg(elasticity = ('elasticity', 'mean')).
summary = pd.merge(summary, sum_elast, how = 'outer')
summary.round(3)
```

```
Out[11]:
```

	product	price	total_sales	wholesale_price	brand	size	market_share	price_p
0	1	3.427	51280	2.187	Tylenol	25	0.145	
1	2	4.946	63604	3.681	Tylenol	50	0.180	
2	3	7.028	41305	5.771	Tylenol	100	0.117	
3	4	2.969	42032	2.027	Advil	25	0.119	
4	5	5.146	27627	3.630	Advil	50	0.078	
5	6	8.158	12703	6.104	Advil	100	0.036	
6	7	2.674	14365	1.851	Bayer	25	0.041	
7	8	3.619	12310	2.438	Bayer	50	0.035	
8	9	3.971	28324	3.710	Bayer	100	0.080	
9	10	1.935	33175	0.910	Store Brand	50	0.094	
10	11	4.427	27208	1.914	Store Brand	100	0.077	

3. New Product Introduction

The chain of stores you have data from is considering introducing a 25 tablet size bottle.

(a) Assume WTP for the new product will be equal to average WTP of the brand name 25 tablet products, minus the “store brand” effect you estimated in the first part above. Assume there are no promotions of the new product. What will be the expected demand for the new product at a price of \$2.00?

```
In [12]: # create a new dataframe with only brand-name 25 tablet products
brand_25 = otc.loc[otc['product'].isin([1, 4, 7])]
```



```
# let WTP = CS + price + store-brand effect
CS = (1 / alpha) * np.log((1 / brand_25['outside_share'].mean()))
price = brand_25['price'].mean()
store_brand_effect = logit_modela_iv.params.store_brand

WTP = CS + price + store_brand_effect
print(f'WTP is {WTP}')
```

WTP is 2.746115238911965

```
In [13]: # pull coefficients for use in calculating shares
beta_prom = logit_modela_iv.params.prom
beta_storebrand = logit_modela_iv.params.store_brand
alpha = logit_modela_iv.params.price

beta_product = {}
for i in product_numbers:
    beta_product[i] = logit_modelb_iv.params.get(f"product_{i}", 1)

beta_fe = pd.Series(beta_product)
```

```
In [14]: ## calculate shares according to formula in the notes
# exp(representative utility for new good)
numerator = np.exp((alpha * 2) + (beta_prom * 0) + (beta_storebrand * 1))

# sum of exp(representative utilities for existing goods)
sum = np.exp((alpha * otc.groupby('product')['price'].mean()) + (otc.groupby

# denominator needs to include utility from new good as well
denominator = 1 + sum.sum() + numerator
share = numerator / denominator
print(f'Share = {share} and average total sales = {otc['total_sales'].mean()})
```

Share = 0.051248896144472686 and average total sales = 104.40501474926253

The new product will have a market share of 0.05. With an average of 104 medication sales per store per week, we can estimate that each store will sell about 5 units of the new medicine per week.

(b) Break down the benefits and costs to the store of introducing this new format (assume wholesale price is \$1.00 and there are no fixed or other variable costs of the new product introduction).

If the new medication costs 1.00 per unit and is sold for 2.00 per unit, the company will make

1.00 in profit on each unit sold. Knowing that each store will sell an average of 5 units per week, the company will make 5.00 in profit across its 82 stores.

4. Information Intervention

The chain of stores you have data from is considering a campaign to help educate customers that there is no efficacy difference between the brand name and "store brand" drugs.

(a) Assume the campaign increases the WTP for the "store brands" by the full absolute value of the "store brand" effect you estimated in the first part. What is the net benefit and cost to the store? To consumers?

```
In [15]: # recalculate WTP without the store-brand effect
WTP_new = WTP - store_brand_effect
print(f'New WTP is {WTP_new}')
```

New WTP is 3.013285068992196

```
In [16]: # recalculate consumer surplus
CS_old = WTP - 2.00
CS_new = WTP_new - 2.00
CS_new, CS_old
print(f'Consumer Surplus has increased from {CS_old} to {CS_new}')
```

Consumer Surplus has increased from 0.746115238911965 to 1.0132850689921962